Primera Suite

Mathematical Reference

v0.3.2

Contents

1 Shared Primitives . . . 2
- 1.1 Luminance . . . 2
- 1.2 HSV Conversion . . . 2
- 1.3 OKLab / OKLCH . . . 2
- 1.4 Soft Gamut Squeeze . . . 3
- 1.5 Smoothstep . . . 3
- 1.6 Skintone Mask . . . 3

2 Transfer Functions . . . 4
- 2.1 ARRI LogC3 . . . 4
- 2.2 ARRI LogC4 . . . 4
- 2.3 RED Log3G10 . . . 4
- 2.4 Sony S-Log3 . . . 4
- 2.5 ACEScct . . . 4
- 2.6 DaVinci Intermediate . . . 4
- 2.7 Kodak Cineon . . . 4

3 Primera — Primary Grading . . . 5
- 3.1 Exposure . . . 5
- 3.2 Black Point . . . 5
- 3.3 Temperature and Tint . . . 5
- 3.4 Contrast . . . 5
- 3.5 Shadow Fill . . . 5
- 3.6 Highlight Gain . . . 5
- 3.7 Roll Off (Soft Highlight) . . . 6
- 3.8 Negative Saturation . . . 6
- 3.9 Positive Saturation . . . 6
- 3.10 Luminance Preservation . . . 6

4 PrimeraHue — Tetrahedral Hue/Density . . . 7
- 4.1 Corner Computation (Rodrigues Rotation) . . . 7
- 4.2 Tetrahedral Interpolation . . . 7
- 4.3 Cinecolor . . . 7
- 4.4 Skintone Protection . . . 8

5 PrimeraSkin — Skintone Sculpting . . . 9
- 5.1 Hue Compression . . . 9
- 5.2 OKLCH Adjustments . . . 9

6 PrimeraSplit — Subtractive Split-Toning . . . 10
- 6.1 Shadow/Highlight Weighting . . . 10
- 6.2 Subtractive Color Model . . . 10
- 6.3 Luminance Compensation . . . 10

1 Shared Primitives

The following operations are shared across multiple tools via code fragments.

1.1 Luminance

Two luminance functions are used throughout. Rec. 709 (BT.709) is the default; Rec. 2020 (BT.2020) is used in PrimeraSplit where DaVinci Wide Gamut is assumed.

$$Y_{709} = 0.2126r + 0.7152g + 0.0722b$$

$$Y_{2020} = 0.2627r + 0.6780g + 0.0593b$$

1.2 HSV Conversion

Standard hexagonal-model conversion between RGB and HSV. Given $C_{\max} = \max(r, g, b)$, $C_{\min} = \min(r, g, b)$, and $\Delta = C_{\max} - C_{\min}$:

$$H = \begin{cases} 0 & \text{if } \Delta = 0 \\ \frac{1}{6} \cdot \frac{g-b}{\Delta} & \text{if } C_{\max} = r \\ \frac{1}{6} \cdot \left(\frac{b-r}{\Delta} + 2\right) & \text{if } C_{\max} = g \\ \frac{1}{6} \cdot \left(\frac{r-g}{\Delta} + 4\right) & \text{if } C_{\max} = b \end{cases}$$

with H wrapped to $[0, 1)$, then:

$$S = \begin{cases} \Delta / C_{\max} & \text{if } C_{\max} > 0 \\ 0 & \text{otherwise} \end{cases}, \quad V = C_{\max}$$

The inverse follows the standard piecewise chroma/secondary construction.

1.3 OKLab / OKLCH

Used by PrimeraSkin. The forward transform maps linear sRGB to OKLab via the M_1 and M_2 matrices from Ottosson (2020):

$$\begin{pmatrix} l \\ m \\ s \end{pmatrix} = M_1 \begin{pmatrix} r \\ g \\ b \end{pmatrix}, \quad M_1 = \begin{pmatrix} 0.4122 & 0.5363 & 0.0514 \\ 0.2119 & 0.6807 & 0.1074 \\ 0.0883 & 0.2817 & 0.6300 \end{pmatrix}$$

Each cone response is cube-rooted (using $\operatorname{sgn}(x)\ |x|^{1/3}$ for negative-safe operation):

$$\begin{pmatrix} l' \\ m' \\ s' \end{pmatrix} = \begin{pmatrix} \sqrt[3]{l} \\ \sqrt[3]{m} \\ \sqrt[3]{s} \end{pmatrix}$$

Then:

$$\begin{pmatrix} L \\ a \\ b \end{pmatrix} = M_2 \begin{pmatrix} l' \\ m' \\ s' \end{pmatrix}, \quad M_2 = \begin{pmatrix} 0.2105 & 0.7936 & -0.0041 \\ 1.9780 & -2.4286 & 0.4506 \\ 0.0259 & 0.7828 & -0.8087 \end{pmatrix}$$

The cylindrical form (OKLCH) is the usual polar decomposition:

$$C = \sqrt{a^2 + b^2}, \quad h = \operatorname{atan2}(b, a)$$

The inverse reconstructs $(a, b) = (C \cos h, C \sin h)$ then reverses through M_2^{-1}, cubing, and M_1^{-1}.

1.4 Soft Gamut Squeeze

Applied per-channel. A piecewise function that compresses the shoulder via tanh and the toe via exponential roll-off, keeping the mid-range on the identity. With knee $k = 0.9$ and range $r = 0.1$ for the shoulder, and toe threshold $t_0 = 0.1$:

$$f(x) = \begin{cases} k + r \cdot \tanh\left(\frac{x-k}{r}\right) & \text{if } x > k \\ t_0 \cdot e^{x/t_0 - 1} & \text{if } x < t_0 \\ x & \text{otherwise} \end{cases}$$

The shoulder asymptotically approaches $k + r = 1.0$; the toe asymptotically approaches 0.

1.5 Smoothstep

The Hermite basis used for all smooth interpolation (shadow/highlight blends, skintone masks, split-toning crossfade):

$$\text{smoothstep}(a, b, x) = t^2(3 - 2t), \quad t = \text{clamp}\left(\frac{x-a}{b-a}, 0, 1\right)$$

1.6 Skintone Mask

Detects skin-colored pixels via an HSV-domain hue gate centered at $H_0 = 28\frac{\circ}{360}^\circ \approx 0.0778$ with half-width $w = H_0 \cdot \rho$ (where ρ is the range parameter, default 1.0):

$$d(h) = \min(|h - H_0|, 1 - |h - H_0|)$$

$$M_{\text{hue}(h)} = 1 - \text{smoothstep}(0, w, d(h))$$

$$M_{\text{sat}(s)} = \text{smoothstep}(0.1, 0.25, s)$$

$$M_{\text{skin}(r,g,b)} = M_{\text{hue}(H)} \cdot M_{\text{sat}(S)}$$

where H and S come from the HSV conversion of (r, g, b).

2 Transfer Functions

All tools that operate in log-space use a common set of encode/decode functions mapping between scene-linear and the selected log encoding. Mid-grey (0.18 linear) is the reference exposure throughout.

Each function below is the *encode* direction (linear → log). The decode is the algebraic inverse.

2.1 ARRI LogC3

$$f(x) = \begin{cases} c \cdot \log_{10}(ax+b) + d & \text{if } x > \text{cut} \\ ex + f & \text{otherwise} \end{cases}$$

with $a = 5.5556$, $b = 0.0523$, $c = 0.2472$, $d = 0.3855$, $e = 5.3677$, $f = 0.0928$, cut $= 0.01059$.

2.2 ARRI LogC4

Let $a = \frac{262144-16}{117.45}$, $b = \frac{1023-95}{1023}$, $c = \frac{95}{1023}$.

$$s = \frac{7 \ln 2 \cdot 2^{7-14c/b}}{ab}, \quad t = \frac{2^{14(-c/b)+6} - 64}{a}$$

$$f(x) = \begin{cases} \dfrac{\log_2(ax+64) - 6}{14} \cdot b + c & \text{if } x \geq t \\ \frac{x-t}{s} & \text{otherwise} \end{cases}$$

2.3 RED Log3G10

$$f(x) = \begin{cases} 0.2243 \cdot \log_{10}(155.9753(x+0.01)+1) & \text{if } x + 0.01 \geq 0 \\ 15.1927(x+0.01) & \text{otherwise} \end{cases}$$

2.4 Sony S-Log3

$$f(x) = \begin{cases} \dfrac{420 + 261.5 \log_{10}\left(\frac{x+0.01}{0.19}\right)}{1023} & \text{if } x \geq 0.01125 \\ \dfrac{171.21x/0.01125 \quad [\text{linear segment}]}{1023} & \text{otherwise} \end{cases}$$

(The linear segment is simplified here; see source for exact coefficients.)

2.5 ACEScct

$$f(x) = \begin{cases} \frac{\log_2(x)+9.72}{17.52} & \text{if } x > 2^{-7} = 0.0078125 \\ 10.5402x + 0.0729 & \text{otherwise} \end{cases}$$

2.6 DaVinci Intermediate

$$f(x) = \begin{cases} C(\log_2(x+A) + B) & \text{if } x > \text{cut}_{\text{lin}} \\ Mx & \text{otherwise} \end{cases}$$

with $A = 0.0075$, $B = 7$, $C = 0.07329$, $M = 10.4443$, $\text{cut}_{\text{lin}} = 0.002624$.

2.7 Kodak Cineon

$$f(x) = \frac{300 \log_{10}(x(1-\delta)+\delta) + 685}{1023}, \quad \delta = 0.0108$$

3 Primera — Primary Grading

The primary tool applies a chain of per-channel operations in a defined order. Let (r, g, b) denote the working pixel value (initially in the selected log encoding).

3.1 Exposure

Scene-linear gain in photographic stops. The pixel is first decoded to linear, scaled, then operations continue in linear through Temp/Tint before re-encoding:

$$(r, g, b)_{\text{lin}} = 2^n \cdot \text{decode}(r, g, b)$$

where n is the exposure parameter in stops.

3.2 Black Point

Smooth compression of the toe region of each channel (in linear), using an exponential roll-off below a knee:

$$\text{bp}(x) = \begin{cases} x - \beta & \text{if } x - \beta \geq \kappa \\ \kappa \cdot e^{(x-\beta)/\kappa - 1} & \text{if } x > 0 \\ x & \text{otherwise} \end{cases}$$

where $\beta = -p_{\text{bp}}$ (the user's Black Point slider) and $\kappa = 0.005$.

3.3 Temperature and Tint

Per-channel linear-light gains. Temperature shifts along the blue–amber axis; tint along green–magenta:

$$r' = r \cdot 2^{\tau}, \quad g' = g \cdot 2^{\varphi}, \quad b' = b \cdot 2^{-\tau}$$

where τ is the temperature parameter and φ is the tint parameter. The pixel is then re-encoded to the working log space.

3.4 Contrast

A pivoted power function that preserves 0, 1, and the pivot point p. The pivot defaults to encode(0.18) (the transfer function's mid-grey) plus a user offset. For input $x \in [0, 1]$:

$$\text{contrast}(x) = \begin{cases} p \cdot \left(\dfrac{x}{p}\right)^{\gamma} & \text{if } x \leq p \\ 1 - (1 - p) \cdot \left(\dfrac{1 - x}{1 - p}\right)^{\gamma} & \text{if } x > p \end{cases}$$

where γ is the contrast parameter. This is symmetric about the pivot: shadows and highlights are stretched/compressed equally.

3.5 Shadow Fill

Below mid-grey $m = \text{encode}(0.18)$, a gain is blended from full effect at black to no effect at mid-grey, using the smoothstep basis as the crossfade:

$$t = \frac{x}{m}, \quad s(t) = t^2(3 - 2t)$$

$$\text{shadow}(x) = x \cdot (G + (1 - G) \cdot s(t)), \quad x < m$$

where $G = 2^{p_{\text{shadows}}}$. Pixels at or above mid-grey pass through unchanged.

3.6 Highlight Gain

The mirror of Shadow Fill, operating between mid-grey and 1.0:

$$t = \frac{x - m}{1 - m}, \quad s(t) = t^2(3 - 2t)$$

$$\text{highlight}(x) = x \cdot (1 + (G - 1) \cdot s(t)), \quad x > m$$

where $G = 2^{p_{\text{highlights}}}$.

3.7 Roll Off (Soft Highlight)

A tanh-based shoulder compression above a variable knee point. The knee descends from 1.0 toward mid-grey as the amount increases; beyond amount = 1.0, a strength multiplier steepens the compression:

$$\kappa = 1 - \min(\alpha, 1) \cdot (1 - m), \quad \sigma = 1 + \max(\alpha - 1, 0) \cdot 2$$

$$\text{rolloff}(x) = \begin{cases} \kappa + (1 - \kappa) \cdot \dfrac{\tanh\left(\dfrac{\sigma(x - \kappa)}{1 - \kappa}\right)}{\sigma} & \text{if } x > \kappa \\ x & \text{otherwise} \end{cases}$$

where α is the Roll Off amount and $m = \text{encode}(0.18)$.

3.8 Negative Saturation

Multiplicative desaturation toward Rec. 709 luminance (log-encoded domain). With saturation factor $s \in [0, 1]$:

$$c' = Y + s(c - Y), \quad c \in \{r, g, b\}$$

where $Y = 0.2126r + 0.7152g + 0.0722b$. At $s = 0$ the image is monochrome.

3.9 Positive Saturation

Operates in HSV space. The S channel is boosted by a power-of-two scale:

$$S' = \min(S \cdot 2^{p_{\text{sat}}}, 1)$$

then converted back to RGB.

3.10 Luminance Preservation

An optional post-step applied after either saturation operation. Scales all channels uniformly to restore pre-adjustment luminance:

$$(r', g', b') = \frac{Y_{\text{before}}}{Y_{\text{after}}} \cdot (r, g, b)$$

4 PrimeraHue — Tetrahedral Hue/Density

Performs per-channel hue rotation and density adjustment via tetrahedral interpolation.

4.1 Corner Computation (Rodrigues Rotation)

Each of the six cube corners (R, G, B, C, M, Y) is rotated around the achromatic axis by a hue angle $\theta = h \cdot 60°$ (where $h \in [-1, 1]$ is the slider value, so ± 1 covers $\pm 60°$). The rotation uses a simplified Rodrigues formula. For a corner with identity color $\boldsymbol{v} = \big(v_r, v_g, v_b\big)$:

$$\mu = \frac{v_r + v_g + v_b}{3}$$

$$\boldsymbol{c} = \boldsymbol{v} - \mu \cdot \mathbf{1}$$

The centred vector is rotated around the $(1, 1, 1)$ axis (with unit direction $\boldsymbol{k} = \frac{1}{\sqrt{3}} \cdot \mathbf{1}$):

$$\boldsymbol{c}' = \boldsymbol{c} \cos\theta + \frac{1}{\sqrt{3}} (\boldsymbol{k} \times \boldsymbol{c}) \sin\theta$$

where the cross-product components reduce to:

$$c'_r = c_r \cos\theta + k\big(c_z - c_y\big) \sin\theta$$

$$c'_g = c_g \cos\theta + k(c_x - c_z) \sin\theta$$

$$c'_b = c_b \cos\theta + k\big(c_y - c_x\big) \sin\theta$$

with $k = \frac{1}{\sqrt{3}} \approx 0.57735$. The density offset d is then added uniformly:

$$\boldsymbol{v}' = \boldsymbol{c}' + \mu \cdot \mathbf{1} + d \cdot \mathbf{1}$$

Density makes the shifted color subjectively more "colorful" without adding energy — it uniformly offsets all three channels at the given corner.

4.2 Tetrahedral Interpolation

The RGB unit cube is decomposed into six tetrahedra by sorting the channel values. Each tetrahedron shares the black corner $(0, 0, 0)$ and white corner $(1, 1, 1)$ and includes two of the six chromatic corners. For a pixel with $r \geq g \geq b$ (the R–Y tetrahedron):

$$\boldsymbol{o} = r \cdot \boldsymbol{R}_c + g \cdot (\boldsymbol{Y}_c - \boldsymbol{R}_c) + b \cdot (\mathbf{1} - \boldsymbol{Y}_c)$$

The five other sort orderings produce analogous expressions using the corresponding pair of adjacent corners. This is the same interpolation structure used in 3D LUT evaluation and described by Yedlin's DisplayPrep approach.

4.3 Cinecolor

A fixed tetrahedral remap that emulates the two-color subtractive process of Cinecolor (c. late 1930s–early 1950s). With blend parameter $t \in [0, 1]$, the chromatic corners are set to:

$$\boldsymbol{R}_c = (1, 0, 0), \quad \boldsymbol{G}_c = (0, 1, t), \quad \boldsymbol{B}_c = (0, 0, 1 - t)$$

$$\boldsymbol{C}_c = (0, 1, 1), \quad \boldsymbol{M}_c = (1, 0, 1 - t), \quad \boldsymbol{Y}_c = (1, 1, t)$$

At $t = 0$ the mapping is identity; at $t = 1$ the blue channel is fully redistributed toward green and away from magenta, emulating orthochromatic film's insensitivity to red light and the resulting blue–orange palette of the duplitized print process.

4.4 Skintone Protection

When Cinecolor is active, an optional holdout matte centered on the skin hue ($\approx 28°$) protects the blue channel from the Cinecolor remap:

$$b_{\text{out}} = b_{\text{cinecolor}}(1 - M_{\text{skin}}) + b_{\text{in}} \cdot M_{\text{skin}}$$

where M_{skin} is the mask defined in Section 1.6.

5 PrimeraSkin — Skintone Sculpting

Dedicated skintone adjustments in OKLCH with an HSV-domain mask.

5.1 Hue Compression

Before the main OKLCH adjustments, an optional hue compression step operates in HSV. Using a slightly broader mask (range +0.43), skin-adjacent hues are pulled toward the median skin hue:

$$d = H - H_0 \quad \text{(wrapped to } (-0.5, 0.5]\text{)}$$

$$d' = d \cdot (1 - \alpha \cdot M^*_{\text{skin}})$$

$$H' = H_0 + d'$$

where $\alpha \in [0, 1]$ is the compression parameter and M^*_{skin} uses the broadened range.

5.2 OKLCH Adjustments

The pixel is converted to OKLab (see Section 1.3) then to cylindrical OKLCH coordinates (L, C, h). Within the skin mask M:

- **Hue rotation:** $h' = h + M \cdot \Delta h$ (Δh in radians, from a $\pm 30°$ slider)
- **Saturation:** $C' = \max(C \cdot (1 + M \cdot \sigma), 0)$ ($\sigma \in [-1, 1]$)
- **Density:** $L' = \max(L \cdot (1 - M \cdot \delta), 0)$ ($\delta \in [-0.5, 0.5]$; positive = darker)

The modified (L', C', h') are converted back to Cartesian OKLab $(L', C' \cos h', C' \sin h')$, then to linear RGB via the inverse OKLab transform. A final per-channel soft squeeze (Section 1.4) contains the gamut.

6 PrimeraSplit — Subtractive Split-Toning

Applies zone-weighted subtractive color adjustments to shadows and highlights independently.

6.1 Shadow/Highlight Weighting

The image luminance (Y_{2020}, Section 1.1) determines the blend between shadow and highlight zones via the smoothstep crossfade:

$$w_{\mathrm{shd}(x)} = 1 - \mathrm{smoothstep}\Big(p - \frac{\tau}{2}, p + \frac{\tau}{2}, x\Big)$$

$$w_{\mathrm{hi}(x)} = \mathrm{smoothstep}\Big(p - \frac{\tau}{2}, p + \frac{\tau}{2}, x\Big)$$

where $p = \mathrm{encode}(0.18) + \mathrm{offset}$ is the effective pivot and τ is the transition softness. The two weights are complementary: $w_{\mathrm{shd}} + w_{\mathrm{hi}} = 1$.

6.2 Subtractive Color Model

Each zone has six adjustment sliders: R, G, B (additive primaries) and C, M, Y (subtractive primaries). The combined adjustment vectors are:

$$\boldsymbol{a}_{\mathrm{rgb}} = w_{\mathrm{shd}} \cdot \boldsymbol{a}_{\mathrm{shd,rgb}} + w_{\mathrm{hi}} \cdot \boldsymbol{a}_{\mathrm{hi,rgb}}$$

$$\boldsymbol{a}_{\mathrm{cmy}} = w_{\mathrm{shd}} \cdot \boldsymbol{a}_{\mathrm{shd,cmy}} + w_{\mathrm{hi}} \cdot \boldsymbol{a}_{\mathrm{hi,cmy}}$$

For each primary, a *positive* slider value subtracts the complementary channels (adding the color by removing its complement), while a *negative* value subtracts the primary itself:

Slider	**Positive (> 0)**	**Negative (< 0)**
Red a_r	$g' = g(1 - a_r), \quad b' = b(1 - a_r)$	$r' = r(1 + a_r)$
Green a_g	$r' = r\big(1 - a_g\big), \quad b' = b\big(1 - a_g\big)$	$g' = g\big(1 + a_g\big)$
Blue a_b	$r' = r(1 - a_b), \quad g' = g(1 - a_b)$	$b' = b(1 + a_b)$
Cyan a_c	$r' = r(1 - a_c)$	$g' = g(1 + a_c), \quad b' = b(1 + a_c)$
Magenta a_m	$g' = g(1 - a_m)$	$r' = r(1 + a_m), \quad b' = b(1 + a_m)$
Yellow a_y	$b' = b\big(1 - a_y\big)$	$r' = r\big(1 + a_y\big), \quad g' = g\big(1 + a_y\big)$

All six adjustments are applied multiplicatively in sequence.

6.3 Luminance Compensation

Subtractive adjustments inherently darken the image. An estimated "subtractive impact" is computed from the adjustment magnitudes (with asymmetric weights: 0.5 for the darkening direction, 0.33 for the complementary), and a compensation gain is derived:

$$\kappa = 1 + \min(2I, 1)$$

$$(r, g, b)_{\mathrm{out}} = (r, g, b) \cdot (1 + (\kappa - 1) \cdot \lambda)$$

where I is the total subtractive impact and $\lambda \in [0, 1]$ is the Preserve Luminance slider.